

Entwurf digitaler Systeme

9. Grundlagen der Applikationsspezifischen Integrierten Schaltungen

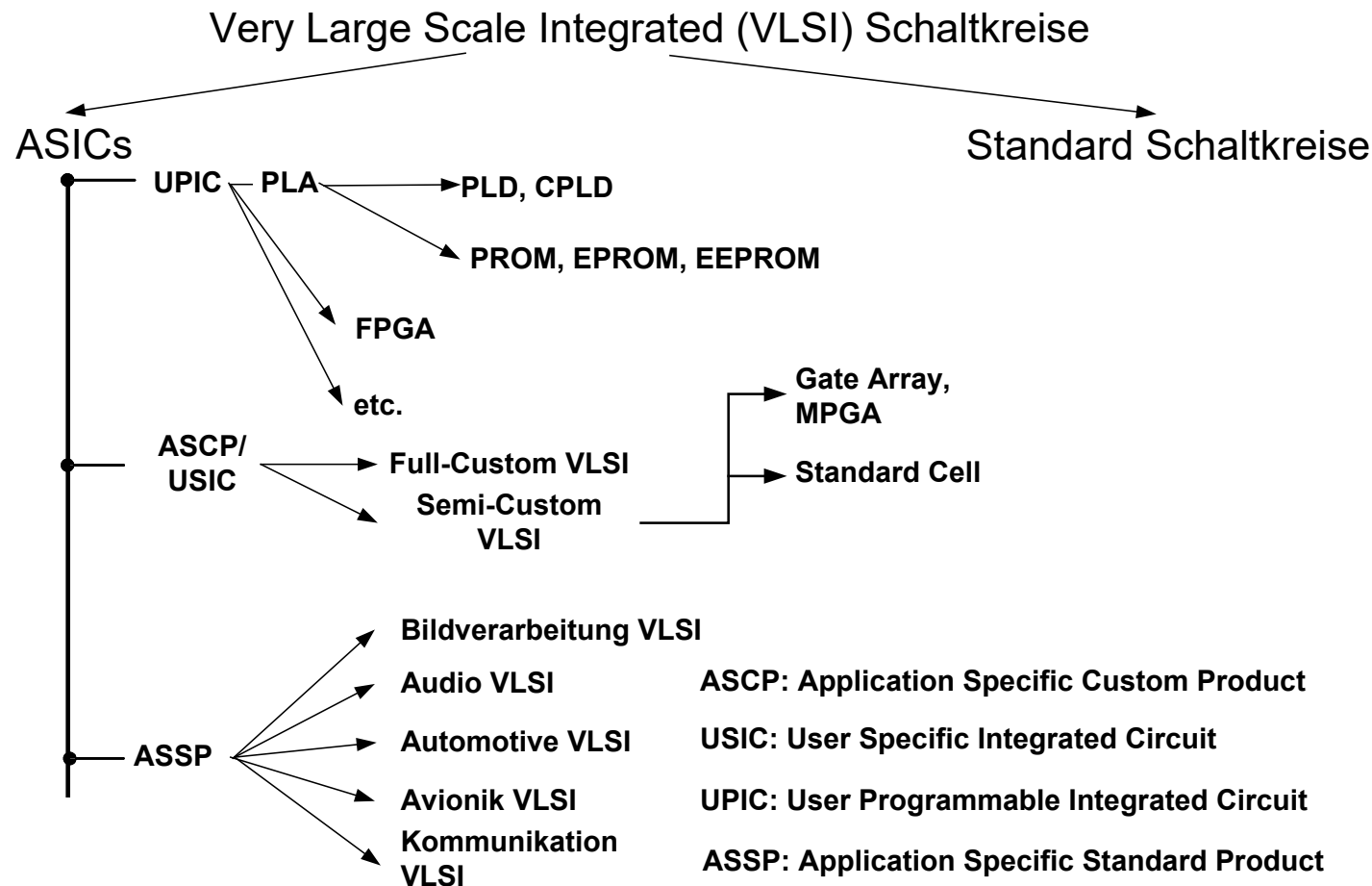
Application Specific Integrated Circuits (ASICs)

9.1 Einführung

- Durch das *Mikroelektronik Dilemma* bleibt nur der Einsatz von Standard-Schaltkreisen (Mikroprozessoren, Mikrocontrollern, Speichern, usw.) oder die Verwendung von applikationsspezifischen integrierten Schaltungen. Die allgemeine Bezeichnung für derartige Bausteine ist der Begriff **ASIC (Application Specific Integrated Circuit, ASIC)**.
- Uns interessiert jedoch zunächst einen Untergruppe dieser Schaltkreise: *Anwenderprogrammierbare integrierte Schaltkreise (User Programmable Integrated Circuit, UPIC)*. Gerade Programmierbare Schaltkreise stellen für den Anwender oft eine preiswerte Alternative für Applikationen dar.
- Einer der wichtigsten Aspekte für den Einsatz anwenderprogrammierbarer Schaltkreise liegt jedoch in den zügigen Entwicklungszeiten begründet (*Time-To-Market-Aspekt*). Durch frühzeitiges Entwickeln und Testen von Prototypen (*Rapid Prototyping, RP*) im Feld lässt sich die Markteinführung von Produkten deutlich beschleunigen.

9.1 Einführung

■ Klassifikation von Schaltkreisen



9.1 Einführung

■ ASSP

- Zu den standardisierten, aber dennoch auf Kundenwünsche zugeschnittenen Schaltkreisen gehören die applikationsspezifischen Kundenprodukte ([Application Specific Standard Product](#), ASSP), die eine spezielle Aufgabe ausführen. Zu nennen sind: WLAN ICs für die Kommunikation, Automobil-Controller mit Busanschluss (CAN), usw.). Diese Bausteine werden standardisiert in relativ großen Stückzahlen hergestellt.

■ ASCP

- Weiterhin werden anwenderspezifische Kunden-ICs ([Application Specific Custom Product](#), ASCP) als so genannte Full-Custom Designs hergestellt. Diese ICs werden nur für einen Kunden entwickelt und hergestellt. Sie gelten als teuer. Davon abweichend lassen sich Semi-Custom Designs realisieren. Bei diesen Bausteinen sind Teile des Schaltkreises vorgefertigt und erlauben nach dem aktuellen Stand der Technik kosteneffektive Lösungen (MPGA).

9.1 Einführung

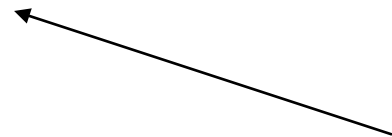
■ UPIC

- Als letzte – uns stark interessierende Gruppe – sind die so genannten anwenderprogrammierbaren integrierten Schaltkreise (UPIC) auszumachen. Diese lassen sich in *Programmable Logic Arrays* (PLA) mit der Untergruppe Programmable Logic Device (PLD), Complex Programmable Logic Device (CPLD) und *Read Only Memory* (ROM) sowie *Field Programmable Gate Arrays* (FPGA) unterteilen.

9.2.1 Programmable Logic Array, PLA

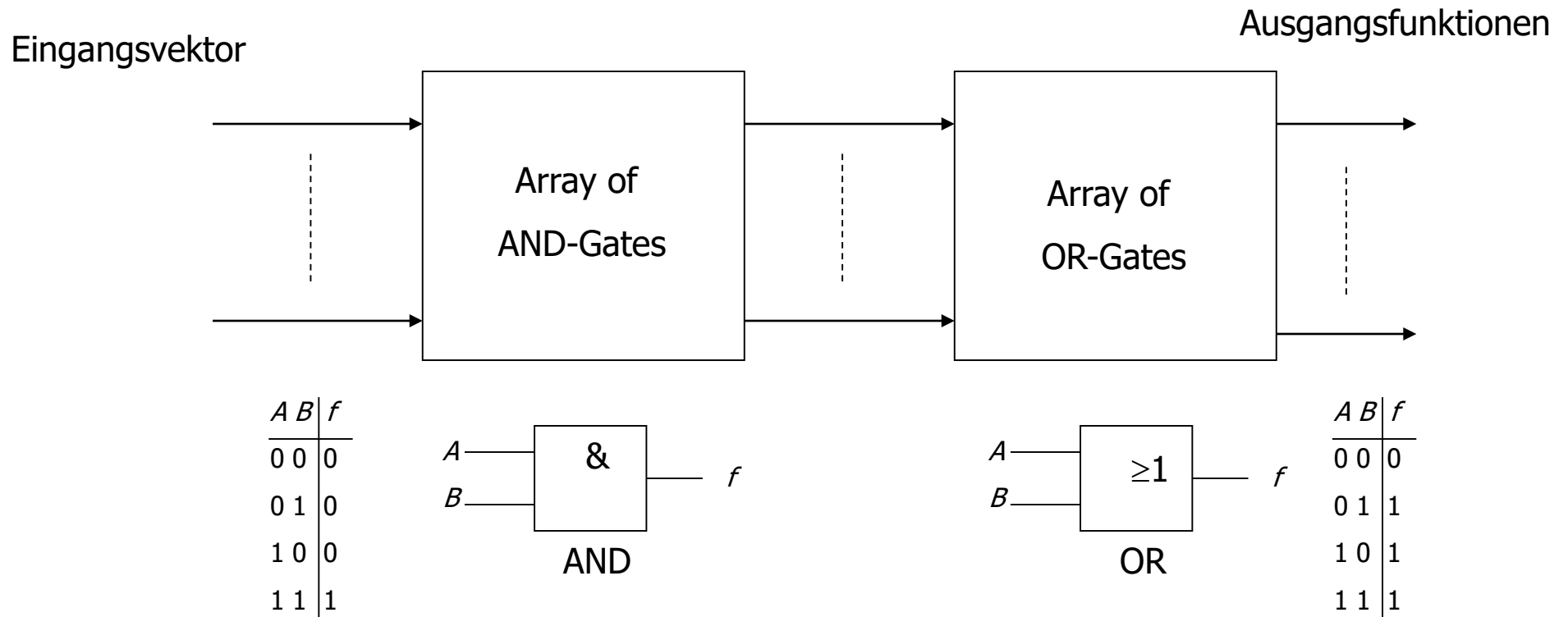
- Die flexibelste programmierbare Schaltung stellt das *Programmable Logic Array* (PLA) dar. Die Prinzipien, die dem PLA zugrunde liegen, stellen die Basis für die in der Praxis eingesetzten Bausteine dar. Das PLA wird heute kaum mehr eingesetzt. Dennoch lassen sich die konzeptionellen Ansätze bei diesem Baustein zielführend erläutern.
- Der Baustein besteht aus einer programmierbaren UND- und ODER-Matrix. Mit dieser lassen sich kombinatorische Funktionen in DNF- bzw. DMF-Form erzeugen:

$$Y = K_0 + K_1 + \cdots + K_{N-1} \quad K_i = x_0 \cdot x_1 \cdot \cdots \cdot x_{M-1}, \quad x \in \{0,1\}$$

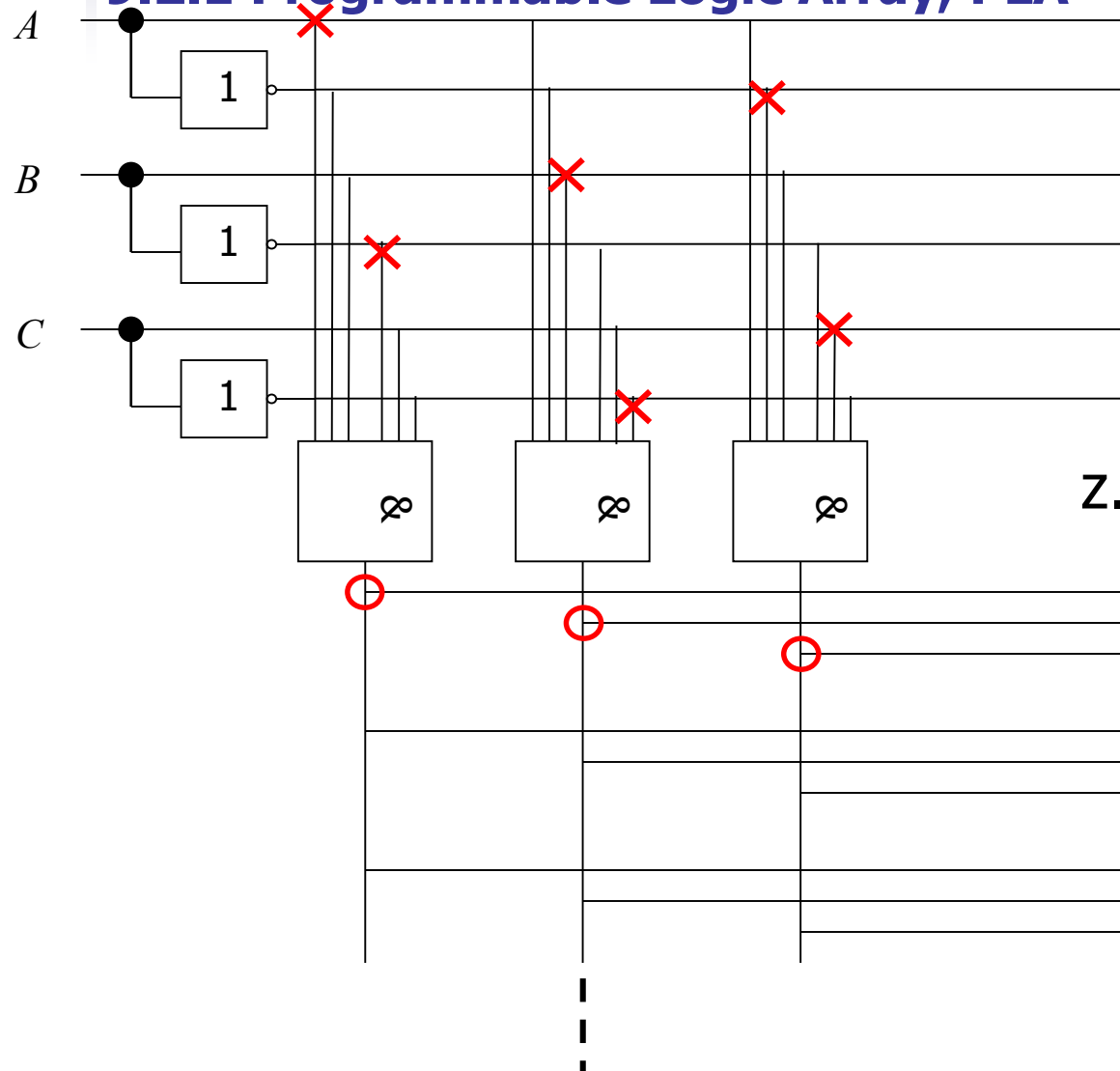


DNF, DMF

9.2.1 Programmable Logic Array, PLA

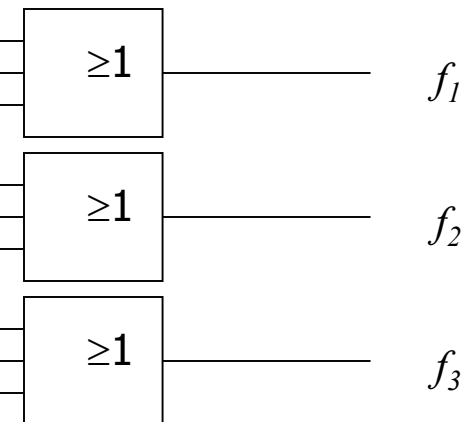


9.2.1 Programmable Logic Array, PLA

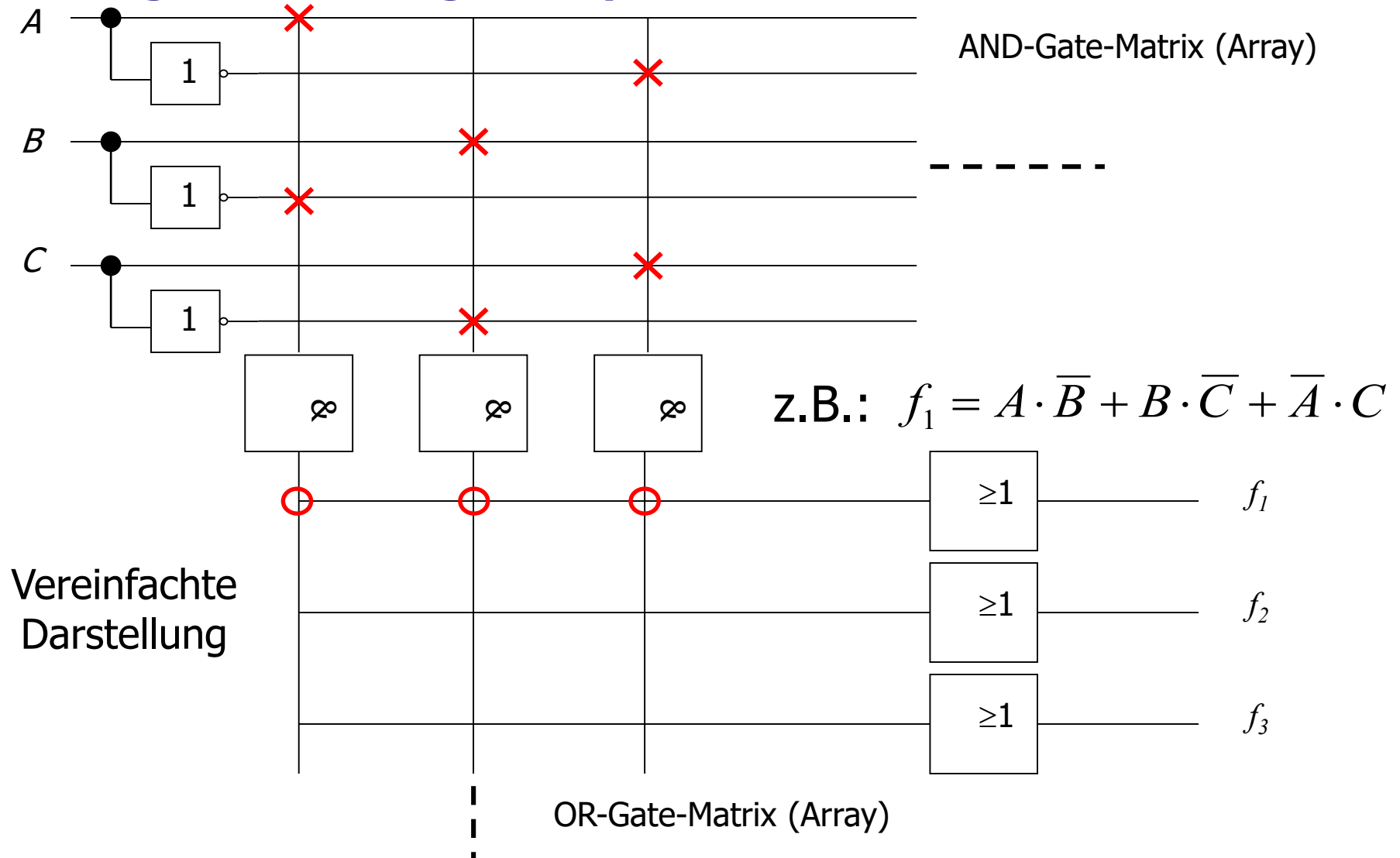


Die Personalisierung wird durch Konfiguration bzw. Programmierung erreicht.

z. B.: $f_1 = A \cdot \bar{B} + B \cdot \bar{C} + \bar{A} \cdot C$



9.2.1 Programmable Logic Array, PLA



9.2.1 Programmable Logic Array, PLA

Ableitung von Varianten

Baustein	AND-Matrix	OR-Matrix	Verknüpfung	Funktion
PLA	programmierbar	programmierbar	POS SOP	KNF DNF
PLD	programmierbar	fest	SOP	DNF
PROM	fest	programmierbar	POS	KNF

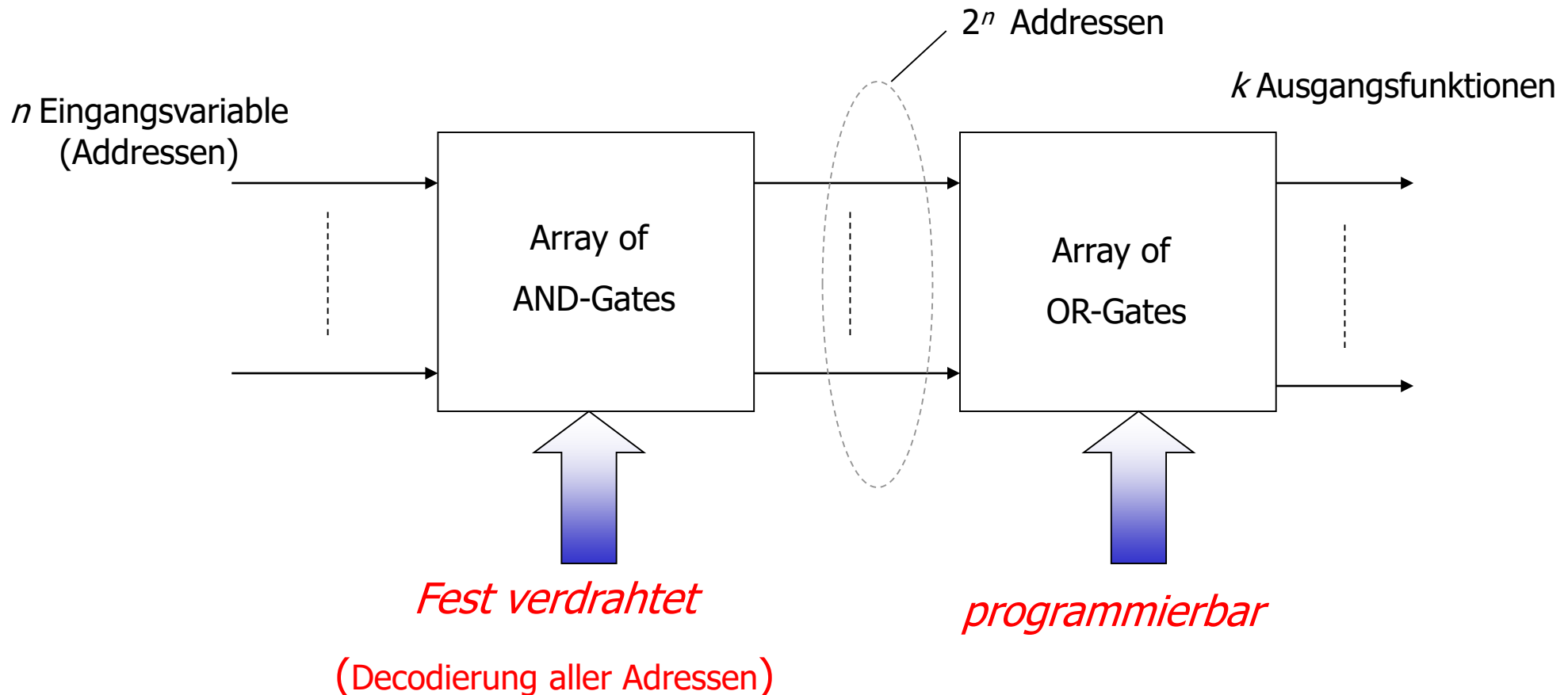
SOP (Sum Of Products): DNF

POS (Product Of Sums): KNF

9.2.1.1 Programmable Read Only Memory, PROM

Bezeichnung: PROM (EPROM, EEPROM):

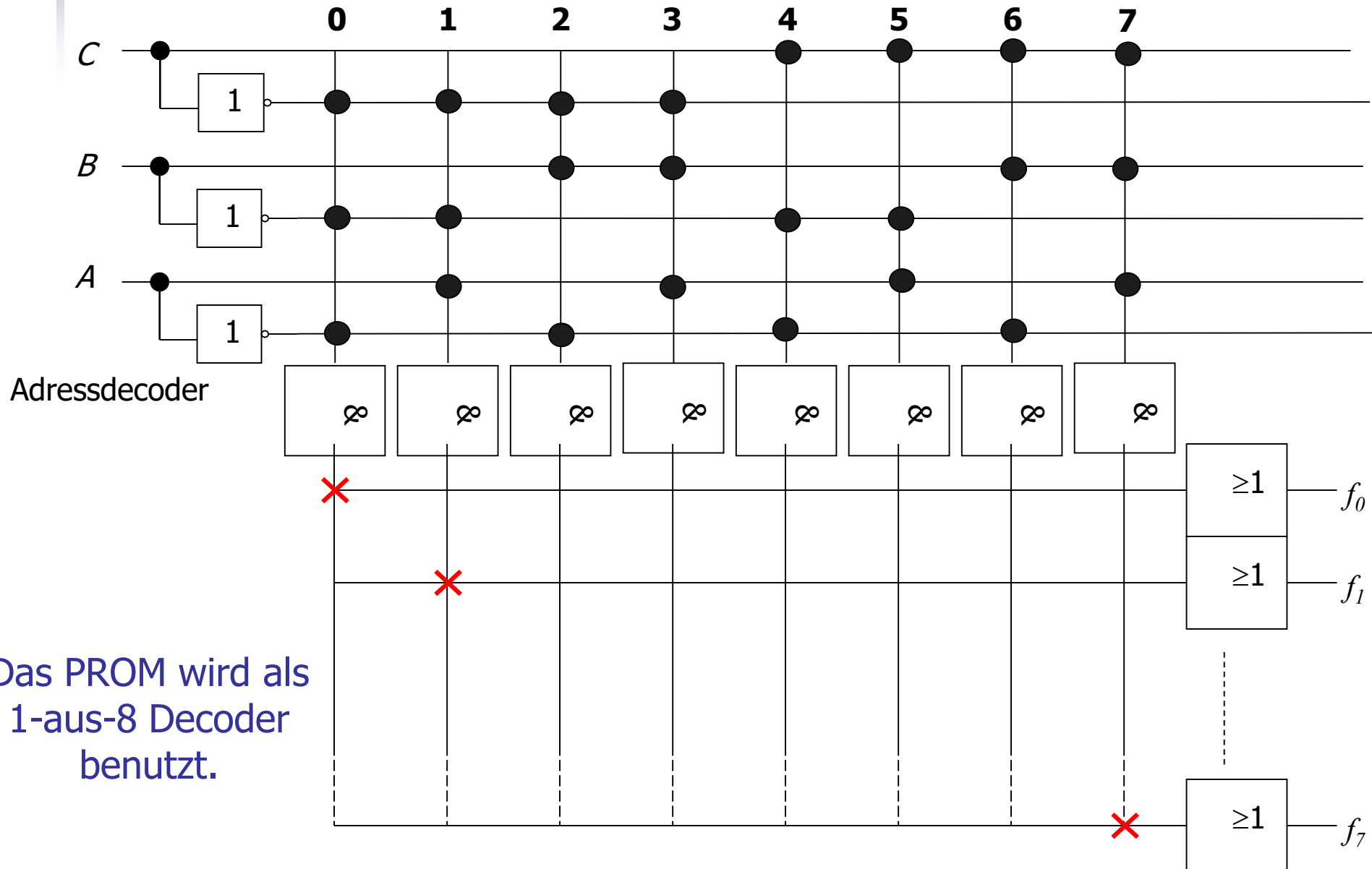
Andere Bezeichnung: **Look-Up-Table (LUT)**



(Decodierung aller Adressen)

Fest verdrahtet

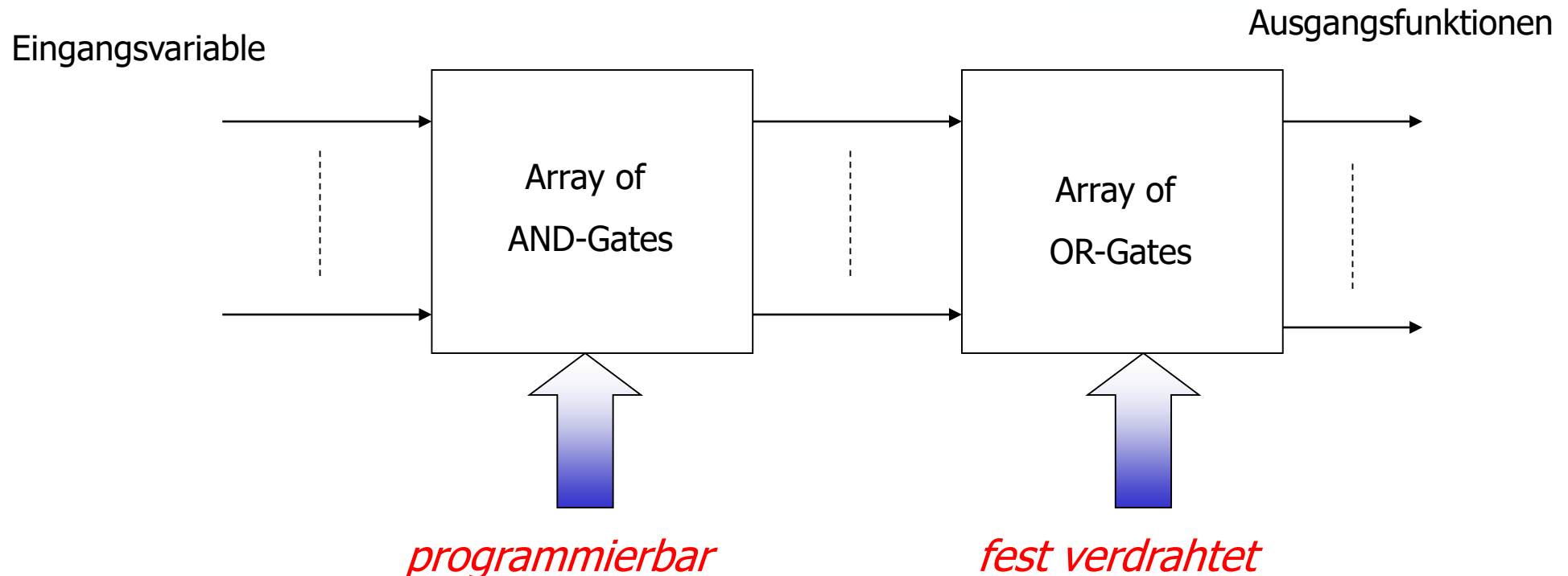
programmierbar



9.2.1.2 Programmable Logic Device, PLD

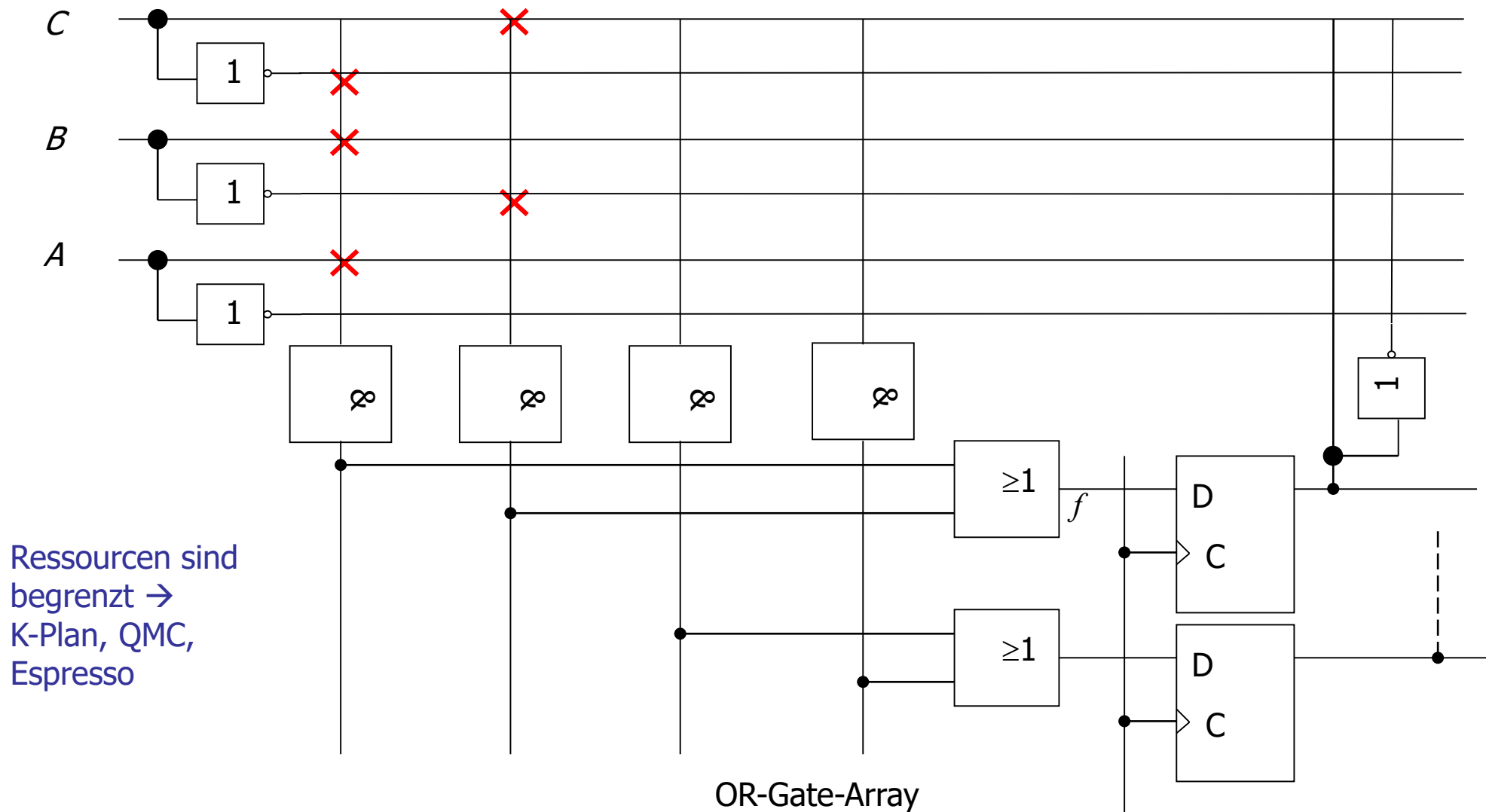
Bezeichnung: (S)PLDs*

Hersteller-Bezeichnung:
PAL (Programmable Array
Logic, AMD) oder GAL (Generic
Array Logic, Lattice)



*) Simple PLD

9.2.1.2 Programmable Logic Device, PLD



Ressourcen sind begrenzt →
K-Plan, QMC,
Espresso

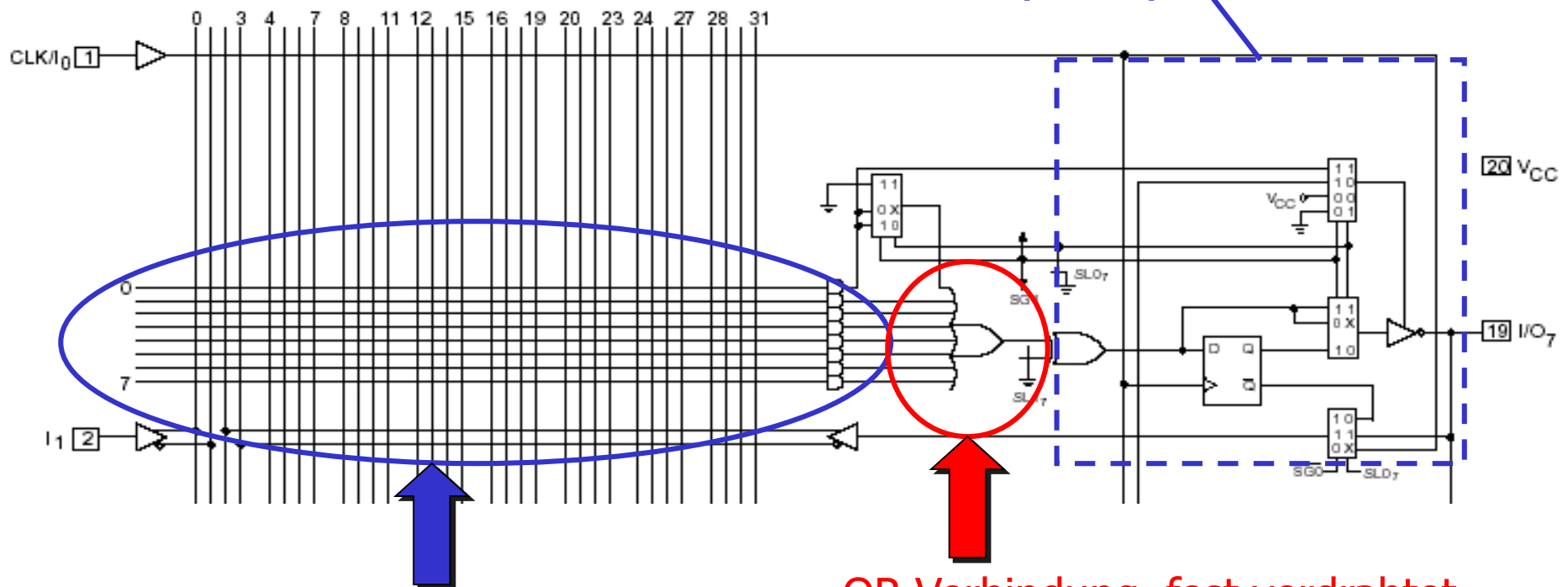
OR-Gate-Array

Clock

9.2.1.2 Programmable Logic Device, PLD

Beispiel: Cypress PAL 22V10

LOGIC DIAGRAM



AND-Array, programmierbar

OR-Verbindung, fest verdrahtet

Quelle: www.cypress.com

9.2.1.2 Programmable Logic Device, PLD

- Zusammenfassend lassen sich für PLDs folgende Aussagen treffen:
- **Vorteile**
 - Sehr gut für zweistufige Logik geeignet.
 - Definierte Laufzeiten innerhalb der Bausteine, da die Struktur homogen ist.
 - Schnell, hohe Taktraten erreichbar.
 - Zu 100 % bei den Herstellern vorgetestet.
 - Garantierte Mindestspeicherzeit.
 - Einfach programmierbar, zum Teil ISP-programmierbar.
- **Nachteile**
 - Nur für einfache Schaltungen verwendbar.
 - Relativ teuer im Vergleich zu anderen (modernerer) Bausteinen.

9.2.1.2 Programmable Logic Device, PLD

■ Anwendungen

- Schnelle kombinatorische Logik.
- Automaten.
- Sequenzer.
- Alle Zähler, Teiler und Automaten sowie kombinatorische Schaltungen, die wir in den Lerneinheiten betrachtet haben, lassen sich mit GALs realisieren.

9.2.1.3 Complex Programmable Logic Device, CPLD

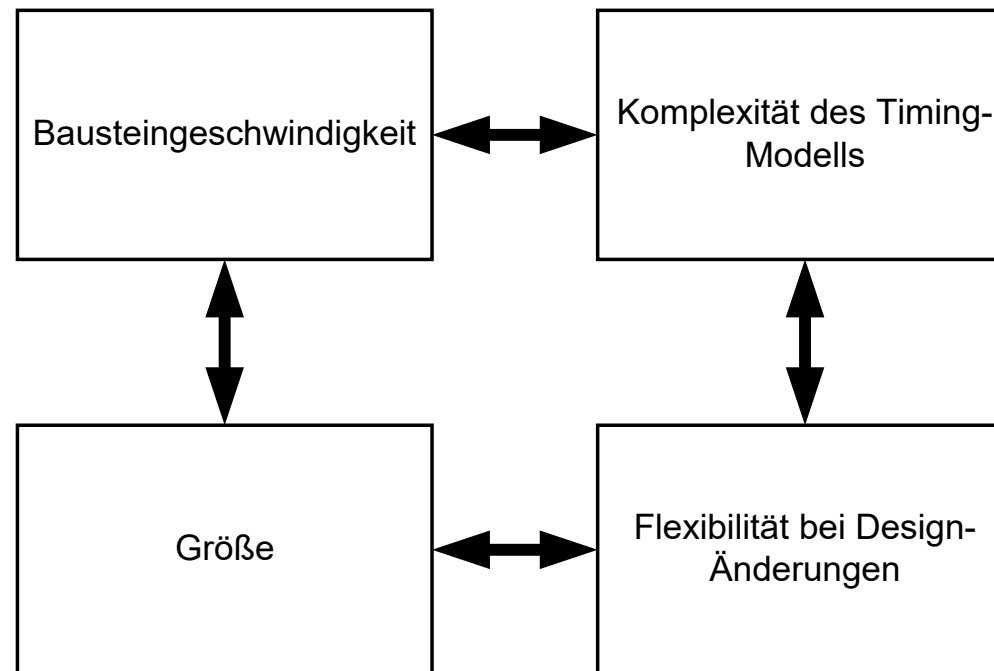
- Da mittels PLDs Schaltungen nur im begrenzten Umfang aufgebaut werden konnten, stellte sich bereits frühzeitig die Frage, wie das erfolgreiche PLD-Konzept erweitert werden kann. Eine einfache Erweiterung der UND-Matrix und der fest verdrahteten ODER-Matrix führte seinerzeit nicht zum Erfolg, da die erweiterten Bausteine aufgrund der langen internen Verbindungswege und großer Streukapazitäten zu langsam wurden. Der Ausweg bestand darin viele GALs in einen Baustein zu integrieren, die geeignet miteinander verbunden waren. Diese speziellen GALs werden üblicherweise als MiniGALs bezeichnet. Der Baustein wird als *Complex Programmable Logic Device* (CPLD) bezeichnet.
- Für entsprechend komplexe Aufgaben können CPLDs verwendet werden, bei denen mehrere LABs (*Logic Array Block*) über eine programmierbare Schaltmatrix miteinander verbunden sind. Die LABs bestehen ihrerseits wiederum aus einer UND-Matrix, Produktterm-Zuordnung und Ausgangsmakrozellen.

9.2.1.3 Complex Programmable Logic Device, CPLD

- Durch den modularen Aufbau erhält man eine Reihe von neuen Möglichkeiten im Vergleich zu einem GAL:
 - Wesentlich größere Schaltungen sind implementierbar.
 - Bausteine mit 16 bis 1024 MiniGALs verfügbar.
 - Mehrere GALs können auf einem Baustein untergebracht werden.
 - Die meisten Bausteine sind im System programmierbar (ISP, IEEE 1532 Standard). Hierfür besitzen die Bausteine in der Regel eine standardisierte Schnittstelle (Joint Test Action Group, JTAG).
 - Test-Schnittstellen sind in den Bausteinen vorhanden (Boundary Scan).

9.2.1.3 Complex Programmable Logic Device, CPLD

- Durch die sehr flexiblen Verbindungsstrukturen ergeben sich jedoch auch Nachteile, die beim CPLD-Entwurf zu berücksichtigen sind. Da sich die Anforderungen bei dem Entwurf teilweise widersprechen, ist es üblich, die Abhängigkeiten in einem so genannten *Magischen Viereck* zu beschreiben.



9.2.1.3 Complex Programmable Logic Device, CPLD

- Zusammenfassend lassen sich für CPLDs folgende Aussagen treffen:
- **Vorteile**
 - Sehr gut für zweistufige Logik geeignet.
 - Definiert berechenbare Laufzeiten innerhalb der Bausteine.
 - Schnell, hohe Taktraten erreichbar (bis 400 MHz).
 - Zu 100% bei den Herstellern vorgetestet.
 - Garantierte Mindestspeicherzeit.
 - ISP-programmierbar, Boundary Scan fähig.
- **Nachteile**
 - Das Magische Viereck muss berücksichtigt werden.
 - Unbenutzte MiniGALs lassen sich nicht aus der Schaltmatrix auskoppeln (erhöhter Stromverbrauch).

9.2.1.3 Complex Programmable Logic Device, CPLD

- **Anwendungen**
 - Schnelle kombinatorische Logik.
 - Komplexe Automaten.
 - Komplexe Sequenzer.
 - Controller.
 - Filter.

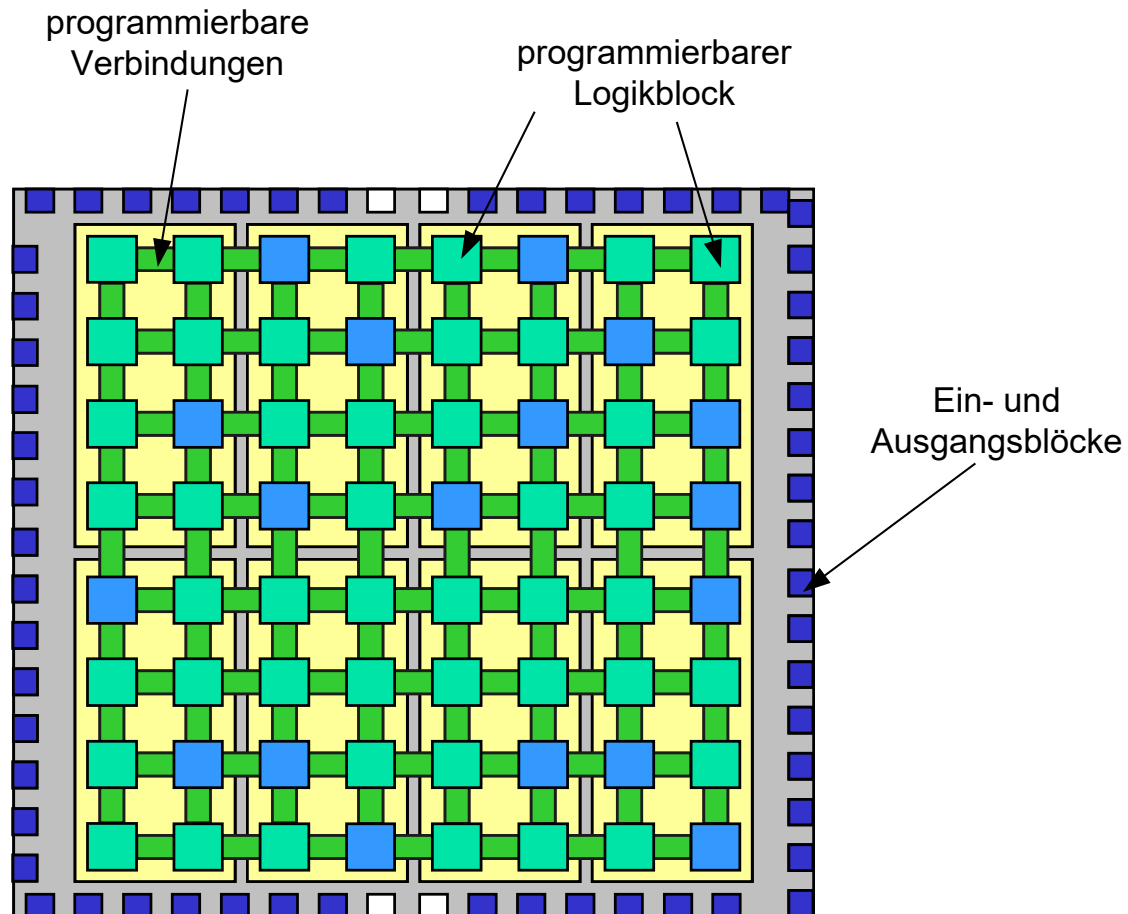
- Da eine große Anzahl verschiedener CPLDs auf dem Markt verfügbar ist, muss im Einzelfall anhand der Anwendung geprüft werden, welcher Baustein der richtige ist.

9.2.2 Field Programmable Gate Array, FPGA

- FPGAs sind mit Hilfe einer großen Anzahl entsprechend einfacher Logikblöcke realisiert, die in der Regel als Feld angeordnet sind.
- Es sind jedoch auch zeilenförmige Strukturen bekannt.
- Zwischen den Blöcken verläuft eine hierarchische Verbindungsstruktur, die zum Ankoppeln von Logikblöcken dient.
- Sowohl die Logikblöcke als auch die Verbindungsstruktur ist programmierbar.
- Mittels der im Vergleich zu den CPLDs feinkörnigeren Struktur der FPGAs entstehen mehr Freiheitsgrade bei der Abbildung einer Logik-Schaltung. Die Hardware-Ressourcen können üblicherweise besser ausgenutzt werden.

9.2.2 Field Programmable Gate Array, FPGA

■ Struktur



9.2.2 Field Programmable Gate Array, FPGA

- Der Entwurf von FPGAs erfolgt heute rechnergestützt mit entsprechenden EDA-Werkzeugen (*Electronic Design Automation*, EDA), da die Komplexität derartiger Schaltungen in vielen Fällen mehrere 100.000 bis zu einige Millionen Gatter beträgt. Deshalb müssen die Entwurfsmethoden Anforderungen entsprechen, die wie folgt charakterisiert werden können:
 - Flexibilität und Wiederverwendbarkeit von Entwürfen
 - Handhabung komplexer Entwürfe (*System-on-programmable-Chip*, SOPC)
 - Entwurfsqualität (*First-time-right*, FTR)
 - Kurze Entwurfszeit bis zum Prototyp (*Rapid Prototyping*, RP)

9.2.2 Field Programmable Gate Array, FPGA

- Die üblichen Programmiersprachen für FPGAs sind VHDL und Verilog.
- Zwischenzeitlich lassen sich die Bausteine (bedingt) auch auf weiter oben angesiedelten Abstraktionsebenen (bedingt) gut programmieren.
- Zu nennen sind u. a. die C-Dialekte *SystemC* und *Handel-C*.
- Weiterhin ist es auch mit dem bekannten Mathematikwerkzeug Matlab/Simulink möglich, entworfene Algorithmen mit den von den Halbleiterherstellern zur Verfügung gestellten Werkzeugen in ein FPGA zu laden.

9.2.2 Field Programmable Gate Array, FPGA

- Das Kernstück vieler FPGAs bilden die Logikblöcke.
- Diese sind von Hersteller zu Hersteller unterschiedlich aufgebaut und benannt (*Logic Element LE, Combinatorial Logic Block CLB*, etc.).
- Das Kernstück der meisten Logikblöcke ist eine so genannte *Look-Up-Table* (LUT).
- Diese LUTs werden auch *Funktionsgeneratoren* genannt. Dabei handelt es sich um eine Tabelle (Speicher) mit üblicherweise 3 oder 4 Eingängen.
- Die Eingänge werden zur Adressierung einer Speicherzelle, die programmierbar ist (Static Random Access Memory, SRAM), verwendet.

9.2.2 Field Programmable Gate Array, FPGA

■ Struktur eines Logikelements (LE)

